先进计算与数字工程研究中心

PATTERN RECOGNITION AND MACHINE LEARNING

Lecture 18 – Deep Learning IV: Sequence Models, Transformers, and New Advances

Chizhi Chris ZHANG

zhangchizhi@ciomp.ac.cn

Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

Three periods and a clear depth boundary

Period	Core content	Learning outcome
1	RNN, BPTT, gradient problems and LSTM gates	Unroll a recurrence and calculate a memory update
2	Q/K/V attention, masks, positions and Transformer blocks	Compute attention and identify allowed information
3	Generative objectives; VAE, GAN, diffusion; state-space orientation	Explain what each model learns and how it samples

Depth

RNN and attention receive worked calculations. Generative families and selective state spaces are a coherent introduction; their full derivations and engineering are follow-up material.

Running language example

Predict the next word from previous words, with no access to the future answer.

A sequence task depends on more than the current input

Task	Input and output	Context requirement
Sentence classification	Words $\rightarrow$ one label	Combine evidence across the sentence
Time-series forecasting	Past measurements $\rightarrow$ future value	Use past only; avoid future leakage
Language modeling	Prefix $\rightarrow$ next-token distribution	Remember useful words in the prefix

- A feedforward model needs a fixed input representation; sequence lengths and relevant distances vary.
- A sequence model defines how information is carried or retrieved across positions.
- The permitted context depends on the task, not only on the architecture name.

Choose the output layout before choosing the sequence model

Task	Output layout	Information allowed
Whole-sentence sentiment	One label after reading the sentence	Entire observed sentence
Label every observed measurement	One output per input position	Depends on offline versus online use
Predict tomorrow's value	Future output from a past prefix	No future measurements

- A hidden state is not automatically the final task output.
- Sequence length and target alignment determine which states feed the output head.
- An offline bidirectional model and an online predictor can use similar layers but solve different access problems.

Autoregressive likelihood makes the prediction task explicit

$$p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t}), \qquad L = -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}).$$

- The model returns a probability distribution over the vocabulary at each position.
- Training uses observed previous tokens and the actual next token as the target.
- Generation repeatedly chooses a token, appends it to the prefix and predicts again.
- The chain rule gives the factorization. RNN or Transformer chooses how to represent the prefix.

A short sentence

Input “I like” predicts the next word. Predicting “cats” must not depend on a representation that already reads “cats”.

Calculate a sequence likelihood from three conditional probabilities

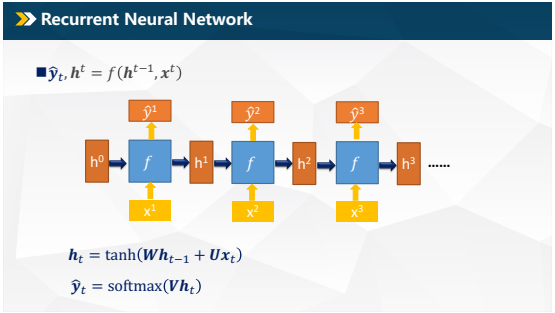
Suppose the observed three-token sequence has conditional probabilities $0.5, 0.25, 0.8$ under its legal prefixes.

$$p(x_1, x_2, x_3) = 0.5(0.25)(0.8) = 0.1.$$

Quantity	Calculation	Meaning
Total negative log-likelihood	$-\log 0.1 \approx 2.303$	Cost for this sequence
Mean per-token loss	$2.303/3 \approx 0.768$	Length-normalized average

- These are conditional probabilities, not independent unconditional token probabilities.
- Summing token losses and averaging them use different scales; comparisons should state the reduction.

An RNN reuses one transition at every time step



Follow the state

Input x_t and previous state h_{t-1} produce h_t. The same transition function is used again at $t+1$.

- Unrolling reveals a chain, not a different network at each time.
- Parameters are shared; states are different.
- Outputs can be produced at every time or only after the sequence.

Memory

The current state summarizes selected information from the prefix.

RNN equations connect current input to shared memory

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h), \qquad o_t = W_o h_t + b_o, \quad p_t = \mathrm{softmax}(o_t).$$

Quantity	Shape	Meaning
x_t	D	Input/embedding at position t
h_t	H	Hidden state
W_x, W_h, W_o	$H\times D,\ H\times H,\ V\times H$	Shared trainable mappings
p_t	V	Probabilities over V output classes/tokens

The state is not a label

It is a learned internal representation. After reading x_t, p_t can predict the next token x_{t+1}.

Count shared RNN parameters without multiplying by sequence length

Let input width $D = 3$, hidden width $H = 4$, output classes $V = 2$.

Parameter	Shape	Number of values
W_x	4×3	12
W_h and b_h	4×4, 4	$16 + 4$
W_o and b_o	2×4, 2	$8 + 2$

Total: $12 + 16 + 4 + 8 + 2 = 42$.

Sharing

A five-position sequence still uses these 42 values. It creates more intermediate states and operations, not five independent parameter sets. Embedding parameters, if present, are additional.

A scalar recurrence shows how old inputs survive

Use a simplified linear recurrence $h_t = 0.5h_{t-1} + x_t$, $h_0 = 0$, and inputs $(1, 2, 0)$.

Time	Input	State calculation	State
1	1	$0.5(0) + 1$	1
2	2	$0.5(1) + 2$	2.5
3	0	$0.5(2.5) + 0$	1.25

$$h_3 = 0.5^2 x_1 + 0.5x_2 + x_3 = 0.25 + 1 + 0 = 1.25.$$

Interpretation

The current state mixes previous inputs with distance-dependent weights. This linear example explains memory flow; an actual RNN uses learned matrices and nonlinear activations.

Check two nonlinear recurrent steps by hand

Use $h_t = \tanh(0.5h_{t-1} + x_t)$, $h_0 = 0$, inputs $(1, -1)$.

Time	Preactivation	Hidden state
1	$0.5(0) + 1 = 1$	$h_1 = \tanh 1 \approx 0.7616$
2	$0.5(0.7616) - 1 \approx -0.6192$	$h_2 \approx -0.5506$

- The second input is negative, but its state also contains positive history from step 1.
- Without that history, $\tanh(-1) \approx -0.7616$.
- Nonlinearity keeps these states bounded; it also contributes derivatives to BPTT.

The same inputs in a different order give a different state

Keep $h_t = 0.5h_{t-1} + x_t$ and $h_0 = 0$.

Input order	State sequence	Final state
$(1, 2, 0)$	$1, 2.5, 1.25$	1.25
$(0, 2, 1)$	$0, 2, 2$	2

- Both sequences contain the same numbers and have the same sum.
- Their final states differ because recent inputs receive different influence.
- An unordered average would lose this particular order information.

Language

"The dog chased the cat" and "The cat chased the dog" use similar words but assign different roles. A sequence representation must support such distinctions.

Exercise: reverse two inputs and compare the final state

Use the linear teaching recurrence $h_t = 0.5h_{t-1} + x_t$, $h_0 = 0$.

1. Calculate states for input order $(1, 3)$.
2. Repeat for the reversed order $(3, 1)$.
3. Compare both final states with the unordered mean of the two inputs.

Quantity	What it preserves	What to explain
Recurrent final state	Unequal temporal weighting	Earlier and later inputs play different roles
Unordered mean	Overall average	The arrangement is lost

Boundary

This recurrence is simplified to make the arithmetic transparent; do not silently substitute it for the tanh model above.

Answer: recent inputs have more influence in this recurrence

$$h_2 = 0.5x_1 + x_2.$$

Input order	States	Final state	Input mean
$(1, 3)$	$1, 3.5$	3.5	2
$(3, 1)$	$3, 2.5$	2.5	2

- The first input is attenuated once; the second enters without an earlier decay.
- Identical sums and means do not imply identical recurrent representations.
- Changing the coefficient to 1 would make this particular linear final state a sum, removing the illustrated order difference.

BPTT is backpropagation on the unrolled computation

$$L = \sum_{t=1}^{T} \ell_t, \qquad \frac{\partial L}{\partial W_h} = \sum_{t=1}^{T} \frac{\partial L}{\partial W_h}\bigg|_{\text{use at } t}.$$

- Unroll forward states over time, retaining the needed intermediate values.
- Propagate each loss backward through the dependencies that influenced it.
- Add gradient contributions from every use of a shared parameter.
- An early state can affect several later losses, so its gradient receives several paths of influence.

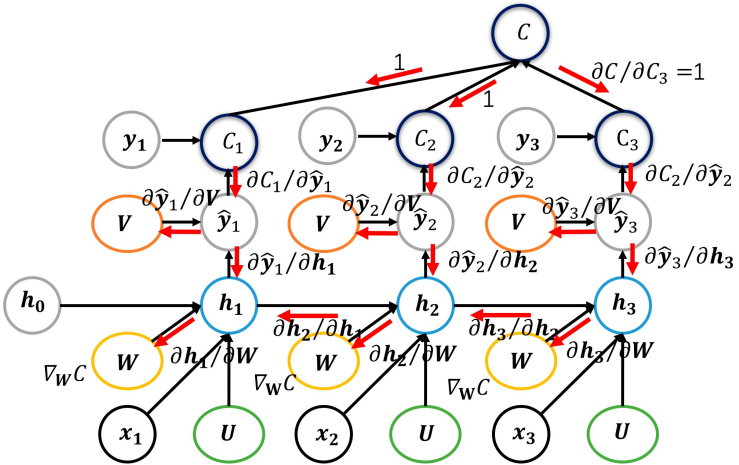
Three positions

A final loss depends on h_3, which depends on h_2, then h_1. Its gradient follows this chain back.

BPTT

Backpropagation through time is the chain rule applied to this expanded graph.

BPTT is backpropagation on the unrolled computation



A shared scalar parameter receives two gradient contributions

Let $h_0 = 1$, $h_1 = wh_0$, $h_2 = wh_1$, and $L = \frac{1}{2}h_2^2$. At $w = 0.5$, $h_1 = 0.5$, $h_2 = 0.25$.

Use of shared w	Contribution path	Gradient contribution
At step 2	$(dL/dh_2)h_1$	$0.25(0.5) = 0.125$
At step 1	$(dL/dh_2)wh_0$	$0.25(0.5)(1) = 0.125$

Total gradient is 0.25. Direct substitution gives $L = \frac{1}{2}w^4$, whose derivative is $2w^3 = 0.25$.

Meaning

Add contributions from both uses before updating the one shared parameter.

Long chains multiply local derivatives

For $s < t$,

$$\frac{\partial h_t}{\partial h_s} = J_t J_{t-1} \cdots J_{s+1}, \qquad J_k = \mathrm{diag}(1 - h_k^2) W_h.$$

- Repeated contractions can make distant gradients tiny: early information is hard to credit.
- Repeated expansion can produce very large gradients and unstable updates.
- The effect depends on matrices, activation derivatives and the trajectory; it is not determined by one weight alone.

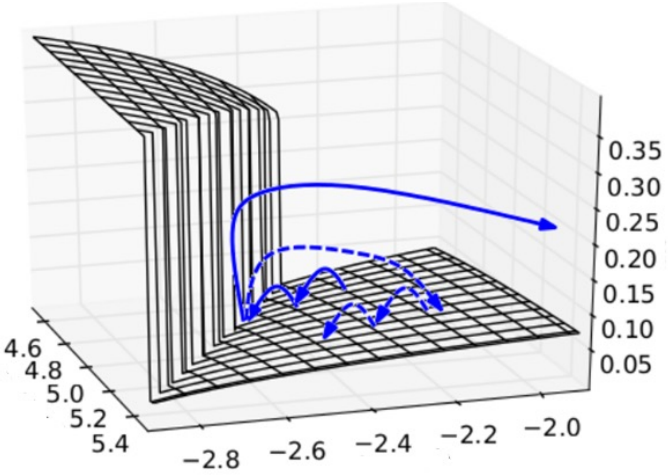
Scalar scale intuition

$0.9^{50} \approx 0.00515$, while $1.1^{50} \approx 117.4$. Small local differences become large over many steps.

Training implication

A signal important far in the past can be difficult to learn even when a short sequence works.

Long chains multiply local derivatives



Compare derivative products over ten and fifty steps

For a scalar path with the same local derivative a at every step, temporal sensitivity is a^K.

Local derivative	Ten steps	Fifty steps
0.8	approximately 0.1074	approximately 1.43×10^{-5}
1.2	approximately 6.1917	approximately 9100

- Repeated modest contraction can strongly suppress distant credit.
- Repeated expansion can overwhelm a stable update scale.
- Real RNNs multiply ordered Jacobian matrices, including activation derivatives. These scalar powers are an explanatory special case, not a general matrix criterion.

Clipping limits explosions; it cannot recover vanished gradients

Clip a gradient by norm threshold c:

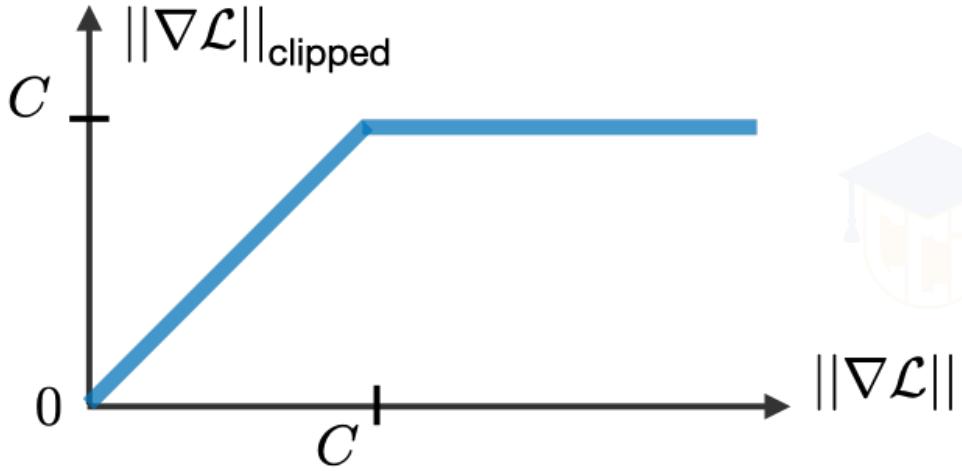
$$g_{\text{clip}} = g \min\left(1, \frac{c}{\|g\|_2}\right).$$

Calculate

For $g = (3, 4)$, $\|g\|_2 = 5$. With $c = 1$, the clipped gradient is $(0.6, 0.8)$: same direction, norm one.

- Apply after backpropagation and before the optimizer update.
- Norm clipping rescales the whole gradient vector; element-wise clipping is a different operation.
- Truncated BPTT limits backward history to save memory, but removes training paths across the truncation boundary.
- Tiny gradients do not become informative just because large ones are clipped.

Clipping limits explosions; it cannot recover vanished gradients



Exercise: compare norm clipping and coordinate clipping

For $g = (6, 8)$, norm is 10. Clip the norm to $c = 5$.

$$g_{\text{norm}} = g(5/10) = (3, 4).$$

Rule	Result	Property
Norm threshold 5	$(3, 4)$	Same direction; norm 5
Each coordinate limited to 5	$(5, 5)$	Changed direction; norm $\sqrt{50}$

At rate 0.1, norm clipping gives parameter change $(-0.3, -0.4)$.

Explain

Both rules limit values, but they are different operations. Neither restores information carried by a gradient that has already vanished.

LSTM separates stored memory from exposed state

- Cell state c_t carries a memory path; hidden state h_t is what the unit exposes for subsequent computation.
- Forget gate f_t chooses how much old memory to retain.
- Input gate i_t chooses how much candidate content $\tilde{c}_t$ to write.
- Output gate o_t chooses how much of the updated memory to expose.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \qquad h_t = o_t \odot \tanh(c_t).$$

Reading a sentence

A relevant topic cue may be retained across uninformative words, then used when predicting a later word. The gates are learned functions, not human-written rules.

LSTM gates are computed from input and previous state

Let $u_t = [h_{t-1}; x_t]$. A standard LSTM without peephole connections is

$$f_t = \sigma(W_f u_t + b_f), \qquad i_t = \sigma(W_i u_t + b_i),$$
$$o_t = \sigma(W_o u_t + b_o), \qquad \tilde{c}_t = \tanh(W_c u_t + b_c),$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \qquad h_t = o_t \odot \tanh(c_t).$$

- Sigmoid gates return values between zero and one.
- The candidate uses tanh so new content can be positive or negative.
- All gates have their own learned parameters, shared across time.

The direct cell path

Its local partial derivative, holding gates fixed, is f_t. A gate near one can support retention, but LSTM does not guarantee unlimited memory.

A sigmoid gate expresses a learned retention fraction

For a gate preactivation s, $f = \sigma(s) = 1/(1 + e^{-s})$.

Preactivation	Forget-gate output	Retained content from $c_{t-1} = 2$
$-\log 9$	0.1	0.2
0	0.5	1.0
$\log 9$	0.9	1.8

- “Forget gate” is a name; its numeric value multiplies the fraction retained.
- The preactivation comes from learned input/state projections.
- Gates act feature by feature and need not be hard zero-or-one choices.

LSTM: retain, write, then expose a numeric memory

For one cell, let $c_{t-1} = 2$, $f_t = 0.8$, $i_t = 0.25$, $\tilde{c}_t = -0.4$ and $o_t = 0.6$.

Operation	Calculation	Result
Retain old memory	0.8×2	1.6
Write new content	$0.25 \times (-0.4)$	-0.1
New cell state	$1.6 - 0.1$	1.5
Exposed hidden state	$0.6 \tanh(1.5)$	approximately 0.543

- Stored memory is 1.5, while the exposed state is smaller and bounded by the output nonlinearity.
- If $f_t = 1$ and $i_t = 0$, this direct update retains the old cell state unchanged.

Exercise: old and new memory can offset each other

For one cell, use $c_{t-1} = 2$, $f_t = 0.2$, $i_t = 0.8$, $\tilde{c}_t = -0.5$, $o_t = 0.5$.

1. Calculate retained old content and newly written content separately.
2. Combine them to get c_t; then compute $h_t = o_t \tanh(c_t)$.
3. If only o_t changes to zero, which stored quantity changes and which remains?

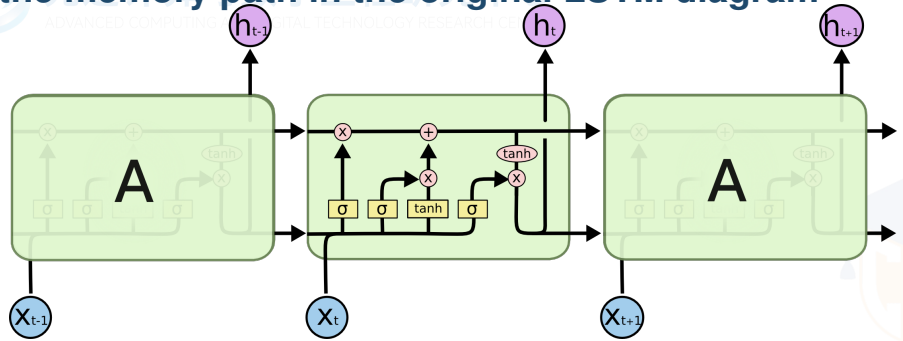
Gate	Controls	Do not confuse with
f_t, i_t	Memory retention and writing	The exposed output fraction
o_t	Exposure of the updated memory	Deleting the cell state itself

Answer: distinguish cancellation from closing the output gate

Operation	Calculation	Result
Retain	$0.2(2)$	0.4
Write	$0.8(-0.5)$	-0.4
Store	$0.4 - 0.4$	$c_t = 0$
Expose	$0.5 \tanh 0$	$h_t = 0$

- Here cell memory becomes zero because two contributions cancel.
- In a different example with $c_t \neq 0$, setting $o_t = 0$ would make $h_t = 0$ while keeping that stored cell state.
- Identical observed outputs can therefore come from different internal mechanisms.

Follow the memory path in the original LSTM diagram



Follow the central cell

The top path retains old memory and adds candidate content. The right branch exposes memory through tanh and an output gate. Adjacent cells share parameters.

Our calculation

Retain 1.6, write -0.1, store $c_t = 1.5$, then expose $h_t \approx 0.543$.

GRU is a compact alternative, not a guaranteed winner

One common GRU convention uses

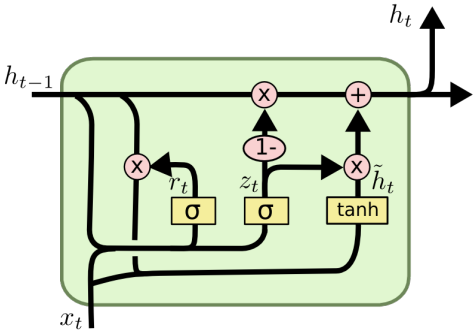
$$\tilde{h}_t = \tanh(W_x x_t + U(r_t \odot h_{t-1}) + b), \qquad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t.$$

- Reset gate r_t controls old-state use in candidate construction.
- Update gate z_t mixes the old state with new candidate content.
- GRU uses one main state rather than separate cell and hidden states, usually with fewer parameters than a matched-size LSTM.
- Libraries and texts sometimes reverse the definition of z; read the formula before interpreting the gate.

Choose empirically

Compare suitable baselines at similar resources. Neither model guarantees stronger memory on every task.

GRU is a compact alternative, not a guaranteed winner



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

GRU: calculate a weighted old-state and candidate mixture

Using this lecture's convention,

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$

Let $h_{t-1} = 0.8$, $\tilde{h}_t = -0.2$, and $z_t = 0.25$:

$$h_t = 0.75(0.8) + 0.25(-0.2) = 0.55.$$

- This gate writes one quarter of the new candidate while retaining three quarters of old state.
- A convention that places z on the old state would assign a different meaning to the same numeric z.
- Reset-gate effects are already included in the supplied candidate here; this calculation isolates the final mixture.

Bidirectionality and truncation change information access

Design	Information path	Appropriate boundary
Unidirectional RNN	State uses current and earlier inputs	Online prediction / causal generation
Bidirectional RNN	Combine forward and backward states	Whole sequence is available
Truncated BPTT	Carry state but backpropagate a shorter segment	Saves memory; limits long-range credit assignment

Avoid leakage

A bidirectional representation of tomorrow's measurements cannot be used to claim a genuine forecast from today's information.

First-period conclusion

RNNs share a transition and compress context into a state. Gates improve retention; computation still follows a temporal dependency chain.

Attention retrieves from visible positions

- An RNN compresses a prefix into a recurrent state; retrieving a specific distant detail can be difficult.
- Attention assigns relevance weights to candidate positions and returns a weighted mixture of their content.
- Self-attention derives queries, keys and values from the same input sequence.
- Cross-attention uses queries from one representation and keys/values from another.

A sentence

To interpret “it”, a representation can retrieve information from relevant earlier nouns instead of relying only on the immediately preceding state.

Relevance is learned

Attention weights are model computations, not a guaranteed human explanation or proof of correct reasoning.

Attention retrieves from visible positions



$\alpha = \mathrm{softmax}(\mathrm{scores})$: weights are nonnegative and sum to one.

Query asks, key matches, value supplies content

For row-vector token representations $X \in \mathbb{R}^{T \times d_{\text{model}}}$,

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

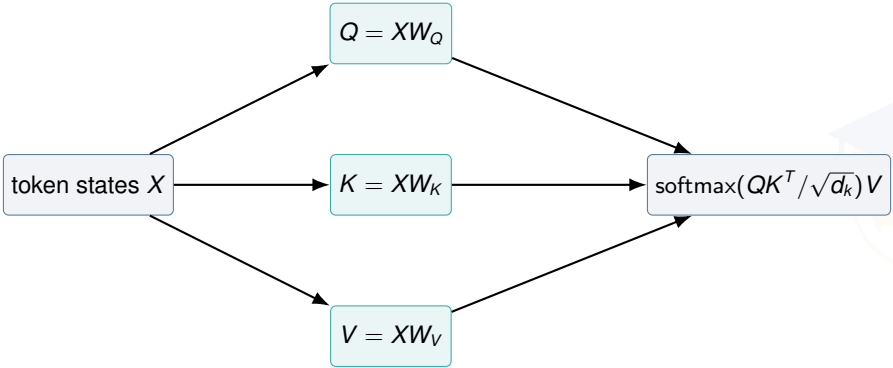
Object	Role	Used in
Query q_i	What position i seeks	Relevance scores against keys
Key k_j	How position j can be matched	$q_i k_j^\top$
Value v_j	Content position j supplies	Weighted sum after softmax

- The three projections are learned; they need not return the same vectors.
- High relevance affects a value's weight. The returned content is the value, not the key.

Consistent convention

Here tokens are matrix rows. A column-vector diagram may place projection matrices on the opposite side.

Query asks, key matches, value supplies content



Check projection shapes before multiplying attention matrices

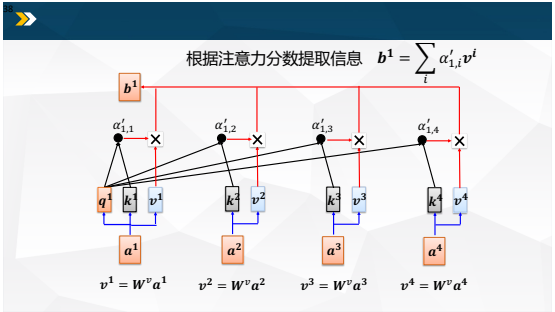
Let $T = 4$, $d_{\text{model}} = 8$, $d_k = 2$, $d_v = 3$.

Object	Shape	Result of the operation
X	4×8	Four input rows
W_Q, W_K	8×2	Q, K: 4×2
W_V	8×3	V: 4×3
$QK^\top$, then AV	4×4, then 4×3	Scores, then retrieved contents

Roles

Query and key widths must support their dot product. Value width can differ; the output carries value features. An output projection later matches the model width when residual addition is needed.

Read one query path in the original attention figure



1. Build one query from position 1.
2. Match it against each allowed key.
3. Apply softmax across those scores.
4. Weight the corresponding values and add them.

One row at a time

Weights for one query sum to one across candidate positions. Another query generally receives a different weight row.

Self-attention

A token can include itself when its mask allows that position.

Exercise: a different query retrieves a different mixture

Use $q = (0, 1)$, keys $k_1 = (1, 0)$, $k_2 = (0, 1)$, $d_k = 2$, values $v_1 = (2, 0)$, $v_2 = (0, 4)$.

1. Compute the scaled scores and softmax weights.
2. Form the weighted value sum.
3. Compare with a query $(1, 0)$ while keeping keys and values fixed.

Answer to check

Scores $(0, 0.7071)$ give weights approximately $(0.3302, 0.6698)$ and output $(0.6604, 2.6792)$.

Meaning

The supplied content did not change. A new query changed which content was emphasized; each query owns its weight row.

Scaled dot-product attention has checkable shapes

$$S = QK^\top/\sqrt{d_k}, \qquad A = \mathrm{softmax}_{\mathrm{row}}(S + M), \qquad O = AV.$$

Tensor	Shape	Meaning
Q, K	$T \times d_k$	Query and key for every position
S, A, M	$T \times T$	Pairwise scores, weights and mask
V	$T \times d_v$	Content vectors
O	$T \times d_v$	Retrieved mixture at each query position

- Softmax acts across keys in each query row.
- $M_{ij} = 0$ allows a position; $M_{ij} = -\infty$ removes it before softmax.

Meaning

Every output row is a weighted sum of allowed value rows.

Attention example: compute scores and row-wise softmax

Take one query $q = (1, 0)$, keys $k_1 = (1, 0)$, $k_2 = (0, 1)$, and $d_k = 2$.

$$s_1 = 1/\sqrt{2} \approx 0.7071, \qquad s_2 = 0.$$

$$a_1 = \frac{e^{0.7071}}{e^{0.7071} + e^{0}} \approx 0.6698, \qquad a_2 \approx 0.3302.$$

- The closer key receives more weight, but the other key is not automatically excluded.
- The weights are nonnegative and sum to one.
- Softmax depends on score differences: subtracting the largest score before exponentiation preserves the weights and improves numerical stability.

Next step

Scores tell us how much content to retrieve. They are not yet the returned content.

Softmax depends on score differences, not a common offset

Score row	Weight row	What changed
$(0.7071, 0)$	approximately $(0.6698, 0.3302)$	Original row
$(0, -0.7071)$	approximately $(0.6698, 0.3302)$	Subtract the same maximum
$(1.4142, 0)$	approximately $(0.8044, 0.1956)$	Multiply gaps by two

$$\frac{e^{s_j - c}}{\sum_k e^{s_k - c}} = \frac{e^{s_j}}{\sum_k e^{s_k}}.$$

Meaning

A common offset cancels from numerator and denominator. Scaling score gaps does not cancel and changes selectivity. Subtracting the row maximum prevents avoidable exponential overflow.

Attention example: retrieve the weighted values

Keep $a = (0.6698, 0.3302)$ and choose values $v_1 = (2, 0)$, $v_2 = (0, 4)$.

$$o = a_1 v_1 + a_2 v_2 \approx 0.6698(2, 0) + 0.3302(0, 4) = (1.3396, 1.3208).$$

Item	Role	Numeric result
Keys	Determine relevance	Scores $0.7071, 0$
Weights	Mix candidates	$0.6698, 0.3302$
Values	Supply content	$(2, 0), (0, 4)$
Output	Retrieved representation	$(1.3396, 1.3208)$

Interpretation

The output combines content from both positions. A larger key score need not imply that every output coordinate is larger.

Equal scores retrieve the mean of the supplied values

If two allowed scores are equal, softmax assigns $(0.5, 0.5)$.

$$0.5(2, 0) + 0.5(0, 4) = (1, 2).$$

Change	Weights	New output
Original values $(2, 0), (0, 4)$	$(0.5, 0.5)$	$(1, 2)$
First value becomes $(4, 0)$	Same weights	$(2, 2)$

- Keys and queries select proportions; values determine the mixed content.
- Equal relevance does not require equal value vectors.
- This mean is a special attention row, not the general rule that all tokens are averaged equally.

The square-root scale prevents dimension-driven score growth

If query and key coordinates are approximately independent, centered and unit-variance, then

$$\mathrm{Var}(qk^\top) \approx d_k, \qquad \mathrm{Var}(qk^\top/\sqrt{d_k}) \approx 1.$$

- Larger dimensions can otherwise produce larger score magnitudes even without greater relevance.
- Extremely separated logits make softmax close to a hard choice and can weaken useful gradients.
- Dividing by $\sqrt{d_k}$ is a scale convention justified by this approximation, not a proof that learned features are independent.

Score contrast

Softmax of $(1, 0)$ is about $(0.731, 0.269)$; $(10, 0)$ is about $(0.99995, 0.00005)$.

A large score gap makes softmax nearly a hard selection

Scores	First weight a_1	Local sensitivity $a_1(1-a_1)$
$(1, 0)$	approximately 0.7311	approximately 0.1966
$(10, 0)$	approximately 0.99995	approximately 0.0000454

For values $(2, 0), (0, 4)$, the second row returns approximately $(1.9999, 0.00018)$.

- The low-scored candidate is almost excluded, but remains a finite softmax term.
- The displayed derivative is one local softmax derivative, not the whole network gradient.
- The square-root scale helps avoid a purely dimension-driven growth of these gaps.

A causal mask forbids future answers before softmax

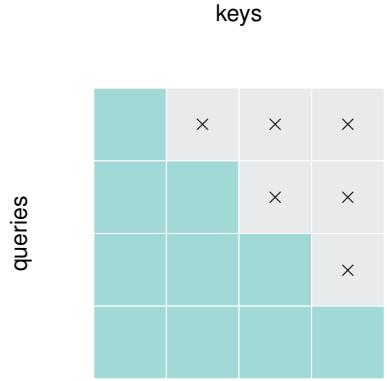
For next-token prediction, query position i may use keys $j \le i$:

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

	Key 1	Key 2	Key 3
Query 1	allowed	blocked	blocked
Query 2	allowed	allowed	blocked
Query 3	allowed	allowed	allowed

- Apply the mask to logits, then normalize the remaining candidates.
- Parallel training still computes all positions, but each row has its own legal prefix.
- Padding masks remove artificial empty positions; causal masks enforce time direction. They solve different problems.

A causal mask forbids future answers before softmax



Each position can use its prefix, including itself.

Padding and causal restrictions remove different invalid information

Input positions are `<BOS>, I, like, <PAD>`.

Query / candidate	Access rule	Why
Query at position 2	Keys 1–2 only	Positions after 2 are future
Query at position 3	Keys 1–3	All three are in its prefix
Padding key at position 4	Block wherever appropriate	It is artificial fill, not observed content
Padding query / target	Handle as invalid for the loss	It should not count as a real token prediction

Implementation boundary

Combine key restrictions before softmax and explicitly handle invalid queries. An all-blocked row is not a valid probability distribution.

Mask example: renormalize the legal candidates

Recall scores $(0.7071, 0)$, weights $(0.6698, 0.3302)$ and values $(2, 0), (0, 4)$.

Access	Softmax weights	Output
Both candidates allowed	$(0.6698, 0.3302)$	$(1.3396, 1.3208)$
Second candidate blocked	$(1, 0)$	$(2, 0)$

- A causal query at position 1 cannot use position 2, even if its value would be helpful.
- The first weight becomes one because normalization uses only the legal candidate.
- Multiplying the old weights by $(1, 0)$ would leave total weight 0.6698 and give the wrong result.

Simple check

Changing a future token must not change a past query's causal-attention output.

Exercise: remove a future key from a three-score row

Scores are $(0, \log 2, \log 4)$; values are $(1, 0), (0, 2), (10, 10)$. Query 2 must block key 3.

Access	Weights	Output to calculate
All keys allowed	$(1/7, 2/7, 4/7)$	Contains future value
Key 3 blocked before softmax	$(1/3, 2/3, 0)$	$(1/3, 4/3)$

- The allowed exponential scores are 1 and 2; their sum is 3.
- Zeroing the old third weight without renormalization would incorrectly give $(1/7, 4/7)$.
- A large future value is unavailable regardless of how useful it appears.

Positions tell attention where tokens occur

Without position information, unmasked self-attention is permutation-equivariant: rearranging input rows rearranges output rows in the same way.

- Token meaning alone cannot identify “first”, “later” or relative distance.
- Add learned or fixed position vectors to token embeddings, or use a position-aware attention mechanism.
- A classic fixed scheme uses

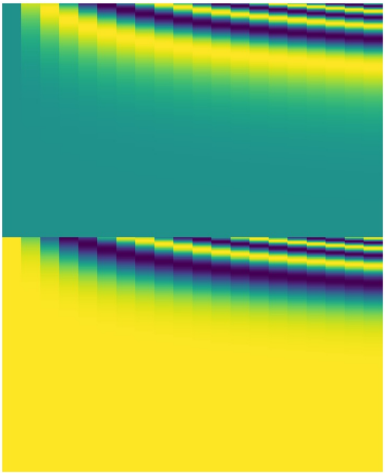
$$PE_{pos,2k} = \sin\left(pos/10000^{2k/d}\right), \quad PE_{pos,2k+1} = \cos\left(pos/10000^{2k/d}\right).$$

- Positions and causal masks are complementary: one expresses order, the other limits access.

Depth boundary

Know why position information is necessary. Rotary and relative-position variants are follow-up designs.

Positions tell attention where tokens occur



Calculate a two-coordinate positional encoding

For the classic scheme with $d = 2$ and $k = 0$, $PE_{pos} = (\sin pos, \cos pos)$.

Position	Position vector	Embedding $(1, 1)$ plus position
0	$(0, 1)$	$(1, 2)$
1	approximately $(0.8415, 0.5403)$	$(1.8415, 1.5403)$
2	approximately $(0.9093, -0.4161)$	$(1.9093, 0.5839)$

- Identical content can become different position-aware inputs.
- Full models use many frequencies or other position designs.
- This encoding supplies order information; a separate mask still enforces future access restrictions.

Position and content jointly determine interpretation

Sentence	Relationship to represent
The dog chased the cat.	Dog is the agent; cat is the object.
The cat chased the dog.	Cat is the agent; dog is the object.

- The same nouns appear, but their roles depend on order and context.
- Position-aware representations allow queries and keys to use this structure.
- A model must still learn the task from data; adding position vectors does not encode all grammar by itself.

Forecasting

A measurement from one hour ago and one year ago may have equal values but different predictive relevance.

Link

Representation includes content and position; the mask includes permitted access.

Multi-head attention combines compatible subspaces

For h heads, commonly $d_k = d_v = d_{\text{model}}/h$:

$$O_r = \text{Attention}(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)}),$$

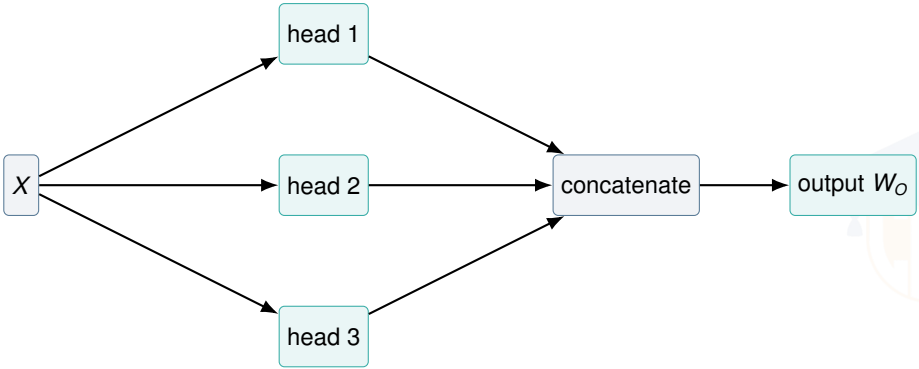
$$O = \text{Concat}(O_1, \ldots, O_h)W_O.$$

- Each head has its own projections and weight matrix over positions.
- With $d_{\text{model}} = 8$ and $h = 2$, each head can return $T \times 4$; concatenation gives $T \times 8$.
- W_O mixes these features back into the model representation.
- Heads may learn different relations, but they are not assigned guaranteed semantic jobs.

Dimensions

Concatenation is not averaging. Its output width must match the output projection and residual path.

Multi-head attention combines compatible subspaces



Each head uses its own query, key and value projections.

Exercise: concatenate two heads and apply an output projection

For one token, head outputs are $O_1 = (1, 2)$ and $O_2 = (3, 4)$. Concatenate them into $c = (1, 2, 3, 4)$.

$$W_O = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}, \qquad cW_O = (4, 6, -2, -2).$$

- The concatenation width is four, not two.
- The supplied 4×4 projection mixes head features while preserving four output coordinates.
- Averaging the two head vectors would give $(2, 3)$ and a different width and meaning.
- A residual path of width four can match the projected output; shape compatibility is deliberate.

The feedforward layer processes each token separately

$$\mathrm{FFN}(x) = \phi(xW_1 + b_1)W_2 + b_2.$$

- Attention mixes information across token positions.
- The FFN transforms features within each position, using the same weights at every position.
- Often the intermediate width exceeds d_{model}, then projects back so residual addition is possible.
- ReLU or GELU is a nonlinear activation; an FFN is not just another positional attention matrix.

Shape example

Input $T \times 8$, first projection 8×32, activation $T \times 32$, second projection 32×8, output $T \times 8$.

Two distinct jobs

Cross-token retrieval and within-token feature transformation alternate in a Transformer.

A tiny FFN shows what the nonlinear stage adds

Use row input $x = (-1, 2)$, identity first projection, ReLU, zero biases and

$$W_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

Stage	Value	Meaning
Input / first projection	$(-1, 2)$	Two features at this position
ReLU	$(0, 2)$	Suppress negative activation
Second projection	$(0, 2)W_2 = (2, -2)$	Recombine nonlinear features

Within a token

This sublayer shares weights across positions but does not directly mix their rows. Its input can already contain context retrieved by attention.

A Transformer block connects retrieval, processing and stable paths

A common pre-norm block is

$$U = X + \mathrm{MHA}(\mathrm{LN}(X)), \qquad Y = U + \mathrm{FFN}(\mathrm{LN}(U)).$$

- Multi-head attention exchanges information among allowed positions.
- FFN transforms each position's representation.
- Residual additions provide a direct path and require matching widths.
- LayerNorm adjusts features within each token; Dropout may be applied in the branches during training.

Variants

The original Transformer used a post-norm arrangement. Pre-norm and post-norm are different designs; keep one convention consistent when reading a diagram or implementing a block.

The residual adds the original state in a pre-norm block

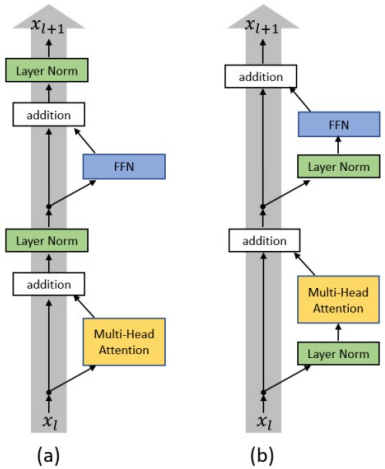
Let $X = (1, 3)$; LayerNorm without affine parameters gives approximately $(-1, 1)$. Suppose the attention branch returns $b = (-0.2, 0.4)$.

Operation	Result	Interpretation
Pre-norm branch input	$\mathrm{LN}(X) = (-1, 1)$	Stabilized branch features
Residual $U = X + b$	$(0.8, 3.4)$	Original state plus learned correction
Mistaken $\mathrm{LN}(X) + b$	$(-1.2, 1.4)$	A different computation

Boundary

The branch output is supplied for illustration. Pre-norm and post-norm layouts are distinct; follow the chosen formula rather than combine pieces from different diagrams.

The residual adds the original state in a pre-norm block



Trace one block using shapes and information access

For a 6-token sequence with $d_{\text{model}} = 8$ and two heads:

Operation	Shape	What changes
Input plus positions	6×8	Content and order encoded
Each head's score matrix	6×6	Pairwise legal relevance
Each head's value mixture	6×4	Retrieved content
Concatenate and project	6×8	Combined head features
Residual, norm, FFN	6×8	Refined representation, same outer shape

Causal block

At query 2, score columns 3–6 are blocked in every head. Increasing layer count does not remove that boundary.

Encoder, decoder and cross-attention serve different tasks

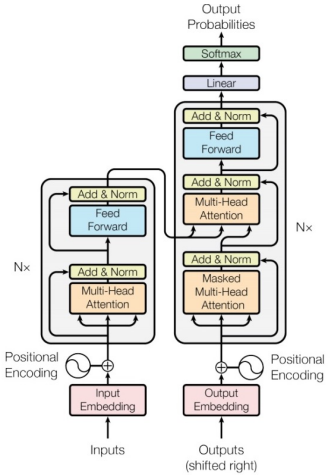
Structure	Context	Example
Encoder	Usually bidirectional self-attention over given input	Sentence classification / representation
Causal decoder	Self-attention over prefix only	Next-token generation
Encoder–decoder	Decoder prefix plus cross-attention to encoded source	Translation / conditional generation

- Cross-attention queries come from decoder states; keys and values come from encoder outputs.
- The source sentence can be fully available, while target generation remains causal.
- Mask choice follows available information at prediction time.

Do not confuse

“Decoder” does not mean it is necessarily the decoder of a VAE; the term depends on the model context.

Encoder, decoder and cross-attention serve different tasks



Parallel teacher forcing and sequential generation are compatible

- In training, shifted observed tokens provide inputs and next tokens provide targets.
- A causal mask lets all target positions be computed in parallel without future leakage.
- In generation, future tokens do not exist. Predict one token, append it, then repeat.
- Sampling temperature rescales logits before softmax; it changes the selection distribution, not trained parameters.

Shifted sequence

Input: `<BOS> I like`. Targets: `I like cats`. The position reading “like” predicts “cats”.

Boundary

The generated prefix can contain earlier model mistakes, unlike the observed training prefix. Fluent output is not proof of factual correctness.

Exercise: align shifted inputs, targets and legal prefixes

Position	Input token	Target token	Allowed input prefix
1	`<BOS>`	`I`	`<BOS>`
2	`I`	`like`	`<BOS>, I`
3	`like`	`cats`	`<BOS>, I, like`

- Each output distribution predicts the next token, not its own current input.
- All three losses can be computed in one masked training pass.
- Allowing target “cats” to enter query 3 would leak the answer.
- During generation, choose a token from the distribution and append it before the next pass.

Long sequences create pairwise cost; caching saves repeated work

- Standard attention forms $T \times T$ pairwise scores per head, giving quadratic sequence-length cost for this part.
- RNNs follow a recurrent chain; token-by-token generation cannot parallelize future unknown outputs.
- A causal Transformer's KV cache stores past keys and values so previous projections need not be recomputed at every generation step.
- The cache uses memory that grows with context length. It does not make all attention cost disappear.

Length effect

Increasing sequence length from 1000 to 2000 multiplies the number of pairwise scores from one million to four million per head.

Transition

Compact state-space models offer another information path. We will return to that after generative objectives.

Generative models learn how samples can vary

- A classifier predicts a label or conditional label distribution from an input.
- A generative model represents a data distribution or a procedure for drawing plausible new samples, often conditional on a label or prompt.
- Autoregressive language generation uses successive conditional distributions.
- Image generation can instead map a latent variable to an image or repeatedly remove noise.

One condition, many outputs

“A dog” can correspond to many poses, colors and backgrounds. The output is not a single fixed target image.

Compare by mechanism

VAE: latent probability model. GAN: learned adversarial feedback. Diffusion: learn noise removal across stages.

A condition restricts a distribution without selecting one fixed sample

Task	Fixed information	Allowed output variation
Classify a dog image	Observed pixels	A label distribution
Generate given “dog”	Class or text condition	Pose, color and background can vary
Unconditional image generation	Learned data distribution	No particular requested label

- Conditioning changes which samples are appropriate; it does not normally remove every source of variation.
- Random latent variables, autoregressive draws or sampling trajectories supply variation differently.
- Judge whether the sample matches its condition separately from whether it looks plausible.

An autoencoder reconstructs; useful random sampling needs more

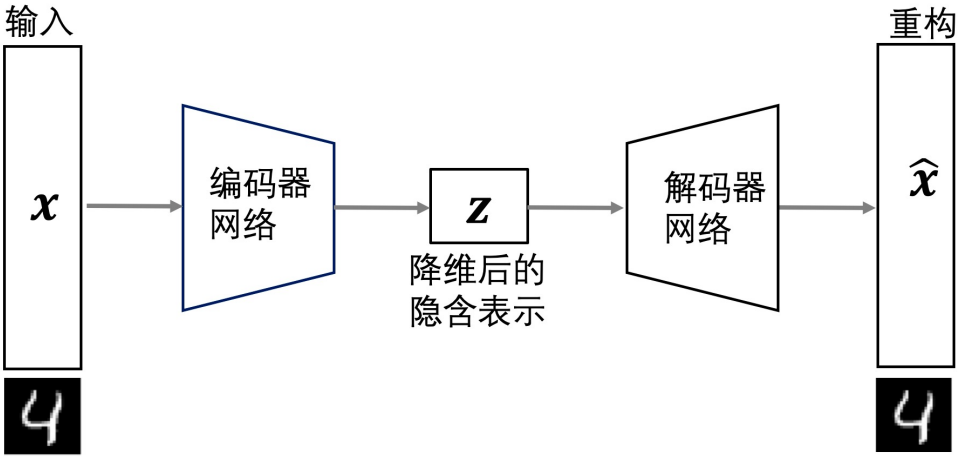
$$z = f_\phi(x), \qquad \hat{x} = g_\theta(z), \qquad L_{\mathrm{AE}} = \|x - \hat{x}\|^2 \quad \text{(one possible loss)}.$$

- The encoder compresses input into a learned code; the decoder reconstructs from it.
- A bottleneck or other constraints encourage useful representation instead of trivial copying.
- Good reconstruction at training codes does not ensure arbitrary random codes decode into plausible samples.
- VAE introduces a probabilistic encoder and a prior to organize the latent space for sampling.

A gap between codes

Two training codes can reconstruct well while an untrained point between or beyond them decodes poorly.

An autoencoder reconstructs; useful random sampling needs more



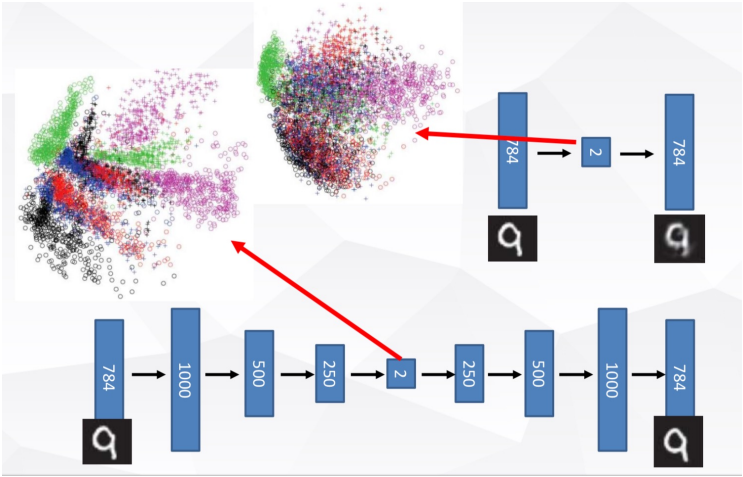
Reconstruction at known codes does not define every latent region

In an illustrative autoencoder, two observed inputs encode to $z_A = 0$ and $z_B = 2$.

Latent point	Available evidence	What may be concluded
0	Reconstructs input A well	The trained code works for A
2	Reconstructs input B well	The trained code works for B
1 or a distant random point	Not specified by these two observations	Plausible decoding is not guaranteed

- The table does not prove interpolation must fail; it shows that two successful reconstructions alone do not prove it succeeds.
- A probabilistic prior and training objective can impose additional organization for sampling.

Reconstruction at known codes does not define every latent region



VAE uses a latent prior and an approximate posterior

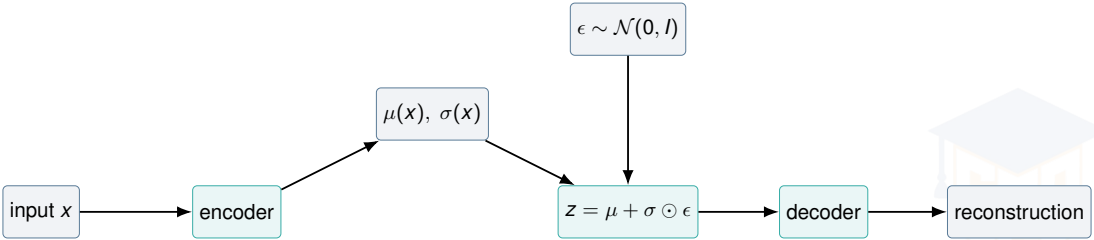
$$p(z) = \mathcal{N}(0, I), \qquad q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \operatorname{diag} \sigma_\phi^2(x)),$$
$$p_\theta(x) = \int p_\theta(x \mid z) p(z)\, dz.$$

- The prior $p(z)$ supplies a simple distribution to sample at generation time.
- The encoder estimates a distribution of latent explanations for an observed x.
- The decoder parameterizes a data likelihood $p_\theta(x \mid z)$, not merely a free-standing code lookup.
- The integral is generally difficult, so training uses an evidence lower bound.

Roles

The encoder is used to infer from observed data; pure generation samples the prior and uses the decoder.

VAE uses a latent prior and an approximate posterior



Generation: draw $z \sim \mathcal{N}(0, I)$ and use the trained decoder.

VAE training and generation follow different routes

Stage	Route	Role of the encoder
Training on observed x	$x \to (\mu, \sigma) \to z \to$ decoder likelihood and KL	Infer a latent distribution for x
Reconstruct an observed input	$x \to q(z \mid x) \to$ decoder	Provide an input-dependent code
Generate from the prior	$z \sim \mathcal{N}(0, I) \to$ decoder	No observed input is required

Explain

The encoder is not automatically needed in every generation call. The sampling prior must be compatible with the latent organization encouraged during training.

The ELBO balances reconstruction and latent organization

$$\log p_\theta(x) \geq \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]}_{\text{reconstruction log-likelihood}} - \underbrace{D_{\mathrm{KL}}(q_\phi(z \mid x) \| p(z))}_{\text{prior matching}}.$$

Minimizing the negative bound means reconstruction negative log-likelihood plus KL.

$D_{\mathrm{KL}} = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1)$ for the diagonal Gaussian encoder and standard normal prior.

- Reconstruction rewards explaining the observed data.
- KL discourages encoder distributions from drifting arbitrarily from the sampling prior.

Not interchangeable

A pixel squared loss corresponds to particular likelihood assumptions; it is not the only reconstruction choice.

Compute Gaussian KL for mean and spread changes

For one latent coordinate,

$$D_{\mathrm{KL}} = \tfrac{1}{2}(\mu^2 + \sigma^2 - \log \sigma^2 - 1).$$

μ	σ^2	KL	Change from the prior
0	1	0	None
1	1	0.5	Mean shift
0	0.25	approximately 0.318	Narrower spread
1	0.25	approximately 0.818	Shift plus narrower spread

Meaning

Both mean and variance matter. A small reconstruction error alone does not describe whether encoder codes occupy a distribution compatible with prior sampling.

VAE: calculate KL and a reparameterized sample

Take $\mu = 1$, $\sigma^2 = 0.25$, hence $\sigma = 0.5$.

$$D_{\mathrm{KL}} = \tfrac{1}{2}[1 + 0.25 - \log(0.25) - 1] \approx 0.818.$$

Use the reparameterization

$$\epsilon \sim \mathcal{N}(0, 1), \qquad z = \mu + \sigma\epsilon.$$

For a draw $\epsilon = 0.4$, $z = 1 + 0.5(0.4) = 1.2$.

- Randomness comes from ϵ, while z is a differentiable transformation of encoder outputs.
- The decoder receives z; gradients can flow back to μ and σ through this expression.
- If $\mu = 0$ and $\sigma^2 = 1$, the KL is exactly zero.

Exercise: sample two codes from one encoder distribution

Use $\mu = 1$, $\sigma^2 = 0.25$, so $\sigma = 0.5$, with draws $\epsilon_1 = -0.4$, $\epsilon_2 = 0.8$.

$$z_r = \mu + \sigma\epsilon_r.$$

1. Calculate z_1 and z_2.
2. Explain why the multiplier is 0.5, not 0.25.
3. Treat each sampled ϵ_r as fixed during differentiation. Find $\partial z_r/\partial\mu$ and $\partial z_r/\partial\sigma$.

Roles

The noise draw supplies randomness; the encoder's mean and standard deviation remain differentiable inputs to the decoder's sampled code.

Answer: sampled codes retain a differentiable dependence on encoder outputs

Draw	Sampled z	$\partial z/\partial \mu$	$\partial z/\partial \sigma$
$\epsilon = -0.4$	$1 + 0.5(-0.4) = 0.8$	1	-0.4
$\epsilon = 0.8$	$1 + 0.5(0.8) = 1.4$	1	0.8

- If the supplied decoder loss gradient is $\partial L/\partial z = 2$, the corresponding μ gradient is 2 in both draws.
- The corresponding σ contributions are -0.8 and 1.6 by the chain rule.
- These are single-draw illustrative contributions, not a universal expected VAE gradient.

Reconstruction and KL reductions change their relative influence

For one illustrative image, let summed reconstruction negative log-likelihood be 120 over 784 pixels, and latent KL be 10.

Convention	Combined number	What must be stated
Reconstruction sum plus KL	$120 + 10 = 130$	Per-image likelihood convention
Reconstruction divided by pixels, same KL	$120/784 + 10 \approx 10.153$	Changed relative weighting

- The two objectives do not differ by one common scale factor.
- A deliberate KL weight can be chosen, but it is a model/training decision rather than an unnoticed reduction change.
- Report the likelihood and reduction definitions before interpreting the balance.

A zero KL alone does not establish a useful input code

Suppose every observed input receives the same encoder distribution $q(z \mid x) = \mathcal{N}(0, I)$.

- The KL to the standard normal prior is zero for every input.
- The sampled code distribution does not distinguish input A from input B.
- Without other input-specific information, this code cannot systematically carry which observed details should be reconstructed.
- The reconstruction term therefore remains essential; minimizing KL alone solves a different, unhelpful task.

Interpretation

VAE training balances data explanation and latent organization. Check both terms and whether the decoder actually uses useful code information.

GAN learns with a generator and a discriminator

- Generator $G(z)$ maps random noise to a sample.
- Discriminator $D(x)$ estimates whether an input is from real data or the generator.
- Alternate updates: improve the discriminator's distinction, then improve the generator using discriminator feedback.

$$\max_D \ \mathbb{E}_{x\sim p_{\text{data}}} \log D(x) + \mathbb{E}_z \log[1 - D(G(z))].$$

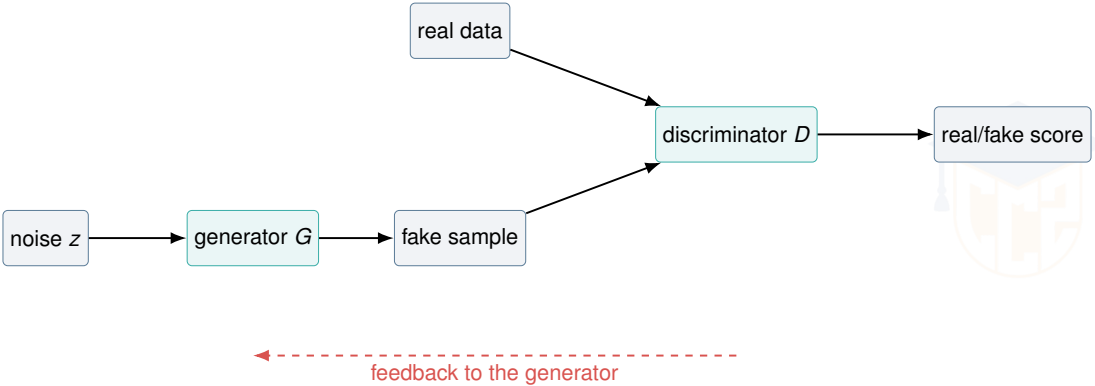
Common generator loss

The non-saturating form is $L_G = -\mathbb{E}_z \log D(G(z))$, minimized over generator parameters.

Interpretation

The judge changes during training. This is an interacting optimization problem, not regression to one fixed target image.

GAN learns with a generator and a discriminator



Calculate discriminator and generator feedback on the same fake score

Use discriminator loss $L_D = -\log D(x) - \log[1 - D(G(z))]$ and non-saturating generator loss $L_G = -\log D(G(z))$.

Real score	Fake score	L_D for this pair	L_G
0.8	0.2	approximately 0.446	approximately 1.609
0.8	0.6	approximately 1.139	approximately 0.511

Opposing feedback

A higher fake-realness score helps the generator's stated objective but makes this discriminator classification loss worse. These are two interacting objectives, not one fixed supervised target.

Separate the two updates without breaking the generator gradient path

Update	Parameters changed	Important gradient route
Discriminator step	Discriminator parameters	Real/fake classification; do not unintentionally update the generator
Generator step	Generator parameters	Backpropagate through discriminator computations into $G(z)$

- Holding discriminator parameters fixed in the generator step does not mean cutting the derivative of its output with respect to generated pixels.
- Detaching generated samples in the generator step would remove the feedback needed to train G.
- The two losses and update boundaries should be explicit.

GAN quality needs diversity, not only convincing examples

- Mode collapse: many latent inputs produce a narrow variety of outputs, even when individual samples look sharp.
- A very strong or weak discriminator can make generator training difficult; game balance is a practical concern.
- A decreasing generator loss alone is not a complete quality measure because the discriminator itself changes.
- Inspect a broad, predeclared sample set rather than select a few attractive examples.

Dog generator

If almost every sample is the same dog pose, sharpness does not demonstrate coverage of the real distribution.

Compare with VAE

VAE optimizes a likelihood bound with latent inference. GAN learns from adversarial discrepancy and usually lacks directly tractable sample likelihood.

Exercise: separate surface variation from meaningful diversity

Illustrative sample set	Observation	What it does not prove
Ten dogs in one pose, different pixel texture	Surface variation exists	Coverage of varied poses and contexts
Ten varied poses, two obvious artifacts	Broader visible variety	Uniformly high realism

- Choose a broad sample set by a declared procedure, rather than selecting only attractive outputs.
- Distinguish realism, diversity and condition match.
- These descriptions do not establish which entire generator is superior; they motivate separate evidence for each claim.

Diffusion creates a known noising training task

Gradually corrupt data using a fixed forward process. Its marginal form is

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

- x_0 is a real training example; t selects a noise level.
- $\bar{\alpha}_t$ controls retained signal versus added noise.
- The training procedure knows the injected noise, supplying a target for a denoising network.
- At generation time, begin near Gaussian noise and apply learned reverse steps.

One shared denoiser

The model receives both x_t and the time/noise level. Separate stages need not mean separate networks.

Diffusion creates a known noising training task



Compare two noise levels for the same clean pixel

Keep clean value $x_0 = 0.8$ and sampled noise $\epsilon = -0.5$.

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon.$$

$\bar{\alpha}_t$	Signal contribution	Noise contribution	Noisy value
0.81	$0.9(0.8) = 0.72$	approximately -0.218	approximately 0.502
0.04	$0.2(0.8) = 0.16$	approximately -0.490	approximately -0.330

Meaning

Lower retained-signal coefficient makes the observation more noise-dominated in this example. The denoiser receives the noise level because the same observed value can have different interpretations at different stages.

Diffusion: a single-pixel calculation explains the target

Let $x_0 = 0.8$, $\bar{\alpha}_t = 0.64$ and the sampled noise be $\epsilon = -0.5$.

$$x_t = 0.8(0.8) + 0.6(-0.5) = 0.34.$$

A noise-prediction network returns $\epsilon_\theta(x_t, t) = -0.4$.

$$L_{\text{noise}} = \big(\epsilon - \epsilon_\theta(x_t, t)\big)^2 = (-0.5 + 0.4)^2 = 0.01.$$

- The target is the injected noise, not a class label.
- An estimate of the clean value from this noise prediction is $(0.34 - 0.6(-0.4))/0.8 = 0.725$.
- Different noise levels teach different denoising conditions; full image prediction uses vectors/tensors.

Exercise: compare clean estimates from two noise predictions

Keep $x_t = 0.34$, $\bar{\alpha}_t = 0.64$, so signal factor is 0.8 and noise factor 0.6.

$$\hat{x}_0 = \frac{x_t - 0.6\epsilon_\theta(x_t, t)}{0.8}.$$

1. Compute the clean estimate when predicted noise is zero.
2. Repeat when predicted noise is -0.5.
3. Compare both estimates with the actual clean value 0.8 and explain the subtraction sign.

Boundary

This is a clean-value estimate at one supplied noise level. Full diffusion generation uses an iterative sampler and may involve additional stochastic updates.

Answer: remove scaled predicted noise before recovering the signal

Predicted noise	Clean-value calculation	Estimate
0	$0.34/0.8$	0.425
-0.4	$[0.34 - 0.6(-0.4)]/0.8$	0.725
-0.5	$[0.34 - 0.6(-0.5)]/0.8$	0.8

- The actual sampled noise was -0.5, so an exact prediction recovers the supplied clean value here.
- The learned model must predict suitable noise from x_t and t; it is not given that answer at generation time.
- A correct toy inversion checks signs and scales, not overall image-generation capability.

Reverse sampling is iterative and condition-dependent

1. Sample an initial noisy state from a simple distribution.
2. At a chosen noise level, use the denoiser and the sampler's update rule.
3. Move to a lower-noise state; repeat until a sample is obtained.

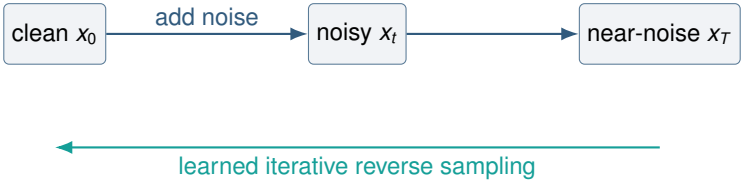
- A conditional denoiser may use a class label or text representation, including cross-attention.
- Prediction parameterization and sampling rule must agree: noise, clean-sample and velocity targets are different conventions.
- Fewer sampling steps can improve speed, but change the quality/cost tradeoff.

Depth boundary

This lecture explains noising, noise prediction and iterative generation. It does not derive every reverse-time or accelerated sampler.

Reverse sampling is iterative and condition-dependent

Training predicts the noise added at a sampled time.



Text conditioning can reuse cross-attention during denoising

As an illustrative conditional design, let N image positions query M encoded prompt tokens.

Object	Shape	Source
Queries	$N \times d_k$	Current noisy image features
Keys / values	$M \times d_k$, $M \times d_v$	Text-encoder features
Scores / returned content	$N \times M$, $N \times d_v$	Text retrieval for each image query

- The full supplied prompt can be available as conditioning information.
- This cross-attention is distinct from causal self-attention over unknown future generated words.
- Conditional architectures vary; the table explains one information path rather than every diffusion implementation.

Compare generative families by training signal and sampling

Family	Training signal	Sampling route	Main issue to inspect
Autoregressive	Next-token likelihood	Token after token	Error accumulation; speed
VAE	Reconstruction likelihood and latent KL	Prior z then decoder	Reconstruction–latent tradeoff
GAN	Adversarial real/fake feedback	Noise z then generator	Stability and mode coverage
Diffusion	Denoising across noise levels	Repeated reverse steps	Quality versus sampling cost

A generated sample is evidence of a procedure

Assess broad quality, diversity, task suitability and memorization with an appropriate evaluation design. A few selected attractive samples do not establish distribution learning.

A state-space recurrence offers compact sequence memory

A discrete linear state-space model can use

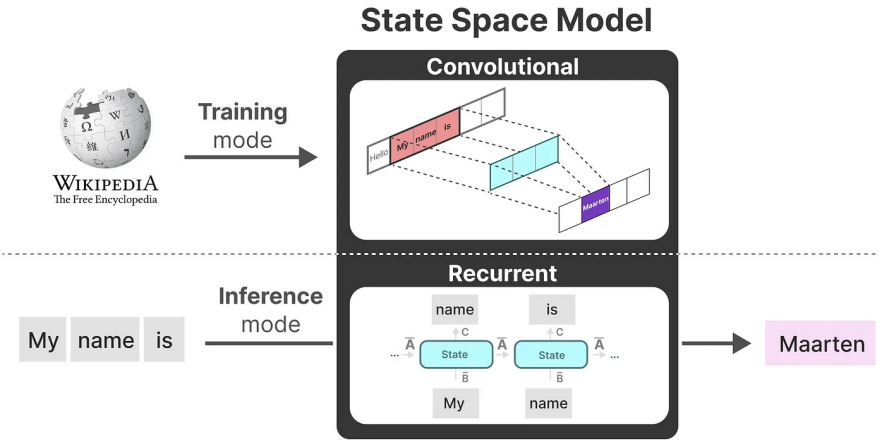
$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \qquad y_t = Ch_t.$$

- A fixed-size state summarizes history instead of storing every pairwise interaction.
- Repeated expansion gives a structured weighting of earlier inputs, related to a convolution for time-invariant parameters.
- For the scalar example $h_t = 0.8h_{t-1} + x_t$, input $(1, 0, 0)$ yields states $(1, 0.8, 0.64)$.
- Fixed dynamics alone do not automatically choose which content matters.

Connection

This resembles the recurrence we calculated early in the lecture. Modern state-space models add parameterization and efficient computation to this basic information path.

A state-space recurrence offers compact sequence memory



Exercise: compare decay memory with content retrieval

Inputs are $(1, 0, 0)$. A scalar state follows $h_t = 0.8h_{t-1} + x_t$, $h_0 = 0$.

Information path	Supplied rule	Final result
Fixed decaying state	States $1, 0.8, 0.64$	0.64
Attention at position 3	Scores $(\log 8, 0, 0)$; weights $(0.8, 0.1, 0.1)$	Weighted value 0.8

- Both use the same available history but assign influence differently.
- The attention weights are supplied for this calculation; the state coefficient is fixed.
- A different output number is not evidence that one architecture is universally better.

Selective state spaces choose what to retain

- A fixed linear recurrence treats timing in a structured but content-independent way.
- Selective state-space models make parts of their dynamics or input/output mapping depend on the current input.
- This can support retaining relevant content and suppressing irrelevant content while maintaining compact state.
- Mamba combines selective dynamics with specialized parallel scan and hardware-aware computation.

Path	Main access pattern	Key tradeoff
Attention	Retrieve from explicit candidate positions	Pairwise interactions and context storage
Recurrent / selective state	Update a compact summary	Information compression and state dynamics

Orientation

This is a conceptual comparison, not a claim that one family universally replaces the other.

Connect the lecture through information and objective

1. RNN carries a shared recurrent state. BPTT trains its use of history; gates help retain and revise memory.
2. Attention computes query–key relevance and mixes values from allowed positions.
3. Positions encode order; masks enforce access; Transformer blocks add FFN, residuals and normalization.
4. Autoregressive, VAE, GAN and diffusion models define different training signals and generation procedures.
5. State-space models revisit compact memory with structured or selective dynamics.

Exit explanation

Compute one attention row; explain why future tokens are masked; describe how a VAE or diffusion model gets its training target and samples a new result.

Next step

Use the task's available information, label/data budget and computation to choose a baseline before comparing larger models.